

AIAC-LAB ASSIGNMENT

Lab 10.2 – Code Review and Quality: Using AI to Improve Code Quality and Readability

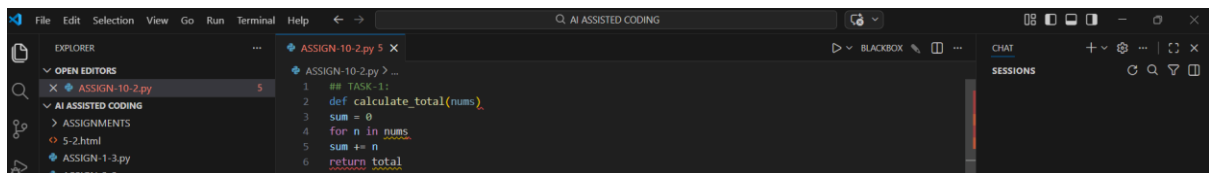
Name: P. Sarika

2403a51l33

B-52

TASK-1: Error Detection and Correction

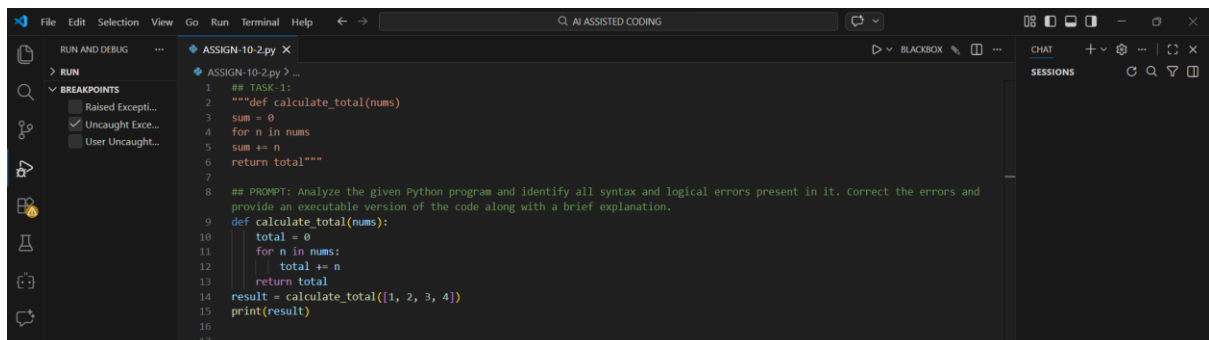
Sample Input Code:



```
1  ## TASK-1:
2  def calculate_total(nums):
3      sum = 0
4      for n in nums
5      sum += n
6      return total
```

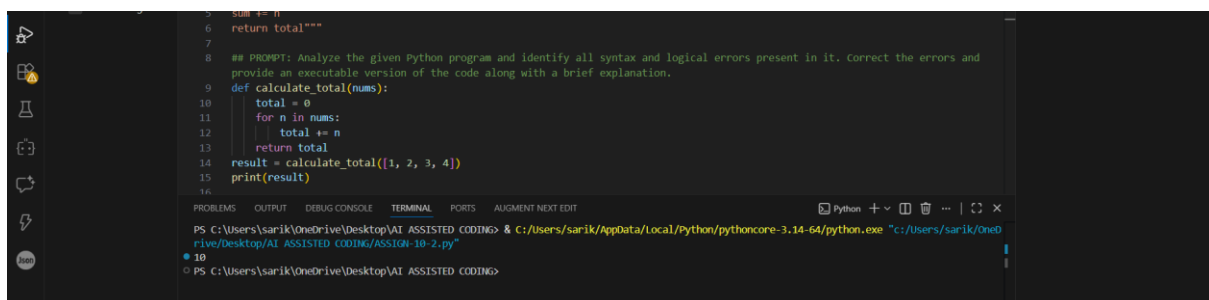
Prompt: Analyse the given Python program and identify all syntax and logical errors present in it. Correct the errors and provide an executable version of the code along with a brief explanation.

Corrected Input Code:



```
1  ## TASK-1:
2  """def calculate_total(nums)
3  sum = 0
4  for n in nums
5  sum += n
6  return total"""
7
8  ## PROMPT: Analyze the given Python program and identify all syntax and logical errors present in it. Correct the errors and
9  provide an executable version of the code along with a brief explanation.
10 def calculate_total(nums):
11     total = 0
12     for n in nums:
13         total += n
14     return total
15 result = calculate_total([1, 2, 3, 4])
16 print(result)
17
```

Output:



```
5  sum += n
6  return total"""
7
8  ## PROMPT: Analyze the given Python program and identify all syntax and logical errors present in it. Correct the errors and
9  provide an executable version of the code along with a brief explanation.
10 def calculate_total(nums):
11     total = 0
12     for n in nums:
13         total += n
14     return total
15 result = calculate_total([1, 2, 3, 4])
16 print(result)
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AUGMENT NEXT EDIT

PS C:\Users\sarika\OneDrive\Desktop\AI ASSISTED CODING> & c:\Users\sarika\AppData\Local\python\pythoncore-3.14-64\python.exe "c:\Users\sarika\OneDrive\Desktop\AI ASSISTED CODING\ASSIGN-10-2.py"

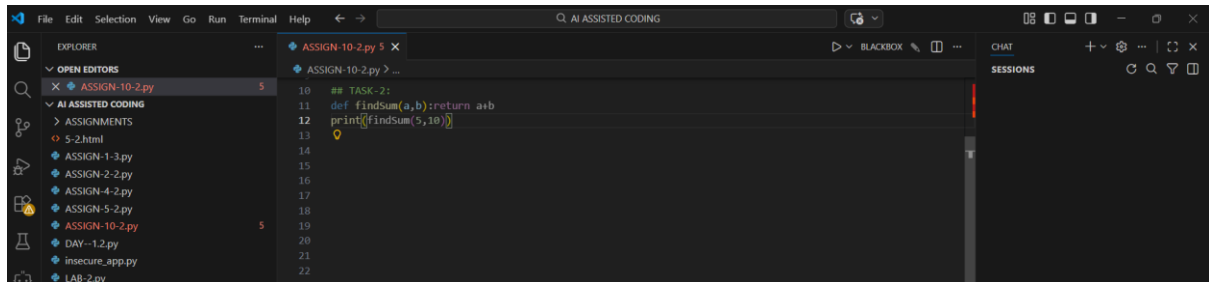
10

PS C:\Users\sarika\OneDrive\Desktop\AI ASSISTED CODING>

Explanation: The original code contained syntax errors such as missing colons and improper indentation, which prevented execution. A logical error was also present where an undefined variable was returned. These issues were corrected to make the function executable and reliable.

TASK-2: Code Style Standardization

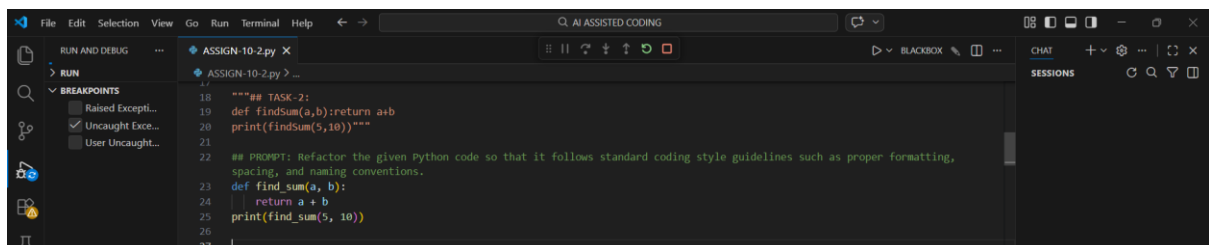
Sample Input Code:



```
10  ## TASK-2:
11  def findSum(a,b):return a+b
12  print(findSum(5,10))
13
14
15
16
17
18
19
20
21
22
```

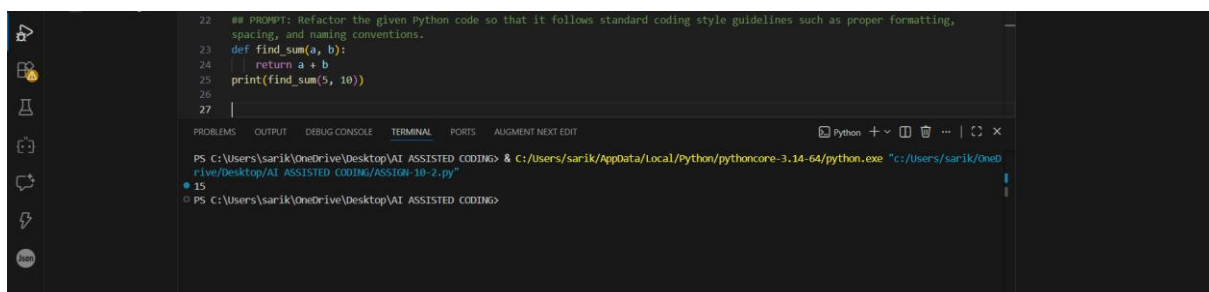
Prompt: Refactor the given Python code so that it follows standard coding style guidelines such as proper formatting, spacing, and naming conventions.

Corrected Input Code:



```
18  """## TASK-2:
19  def findSum(a,b):return a+b
20  print(findSum(5,10))"""
21
22  ## PROMPT: Refactor the given Python code so that it follows standard coding style guidelines such as proper formatting,
23  spacing, and naming conventions.
24  def find_sum(a, b):
25  |   return a + b
26  print(find_sum(5, 10))
27
```

Output:



```
22  ## PROMPT: Refactor the given Python code so that it follows standard coding style guidelines such as proper formatting,
23  spacing, and naming conventions.
24  def find_sum(a, b):
25  |   return a + b
26  print(find_sum(5, 10))
27

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AUGMENT NEXT EDIT

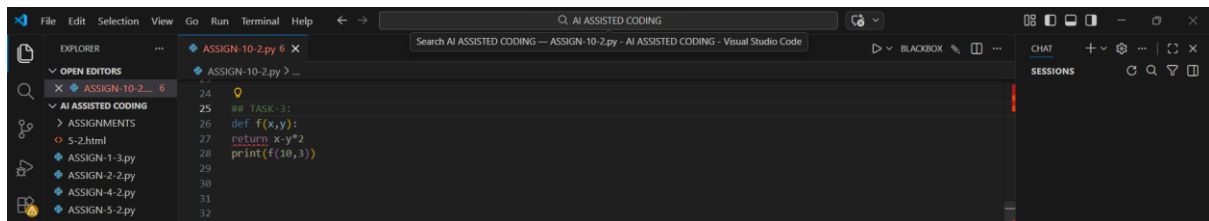
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\Users\sarik\OneD
rive\Desktop\AI ASSISTED CODING\ASSISTED-10-2.py"
15
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation: The code was reformatted to follow Python's PEP 8 style guidelines by improving spacing, naming conventions, and structure. These

changes enhance readability and make the code easier to maintain without altering its functionality.

TASK-3: Code Clarity Improvement

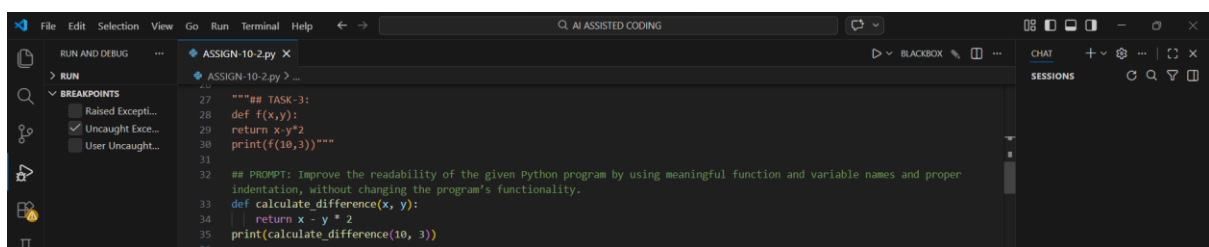
Sample Input Code:



```
24
25 """ TASK-3:
26 def f(x,y):
27     return x-y*2
28     print(f(10,3))
29
30
31
32
```

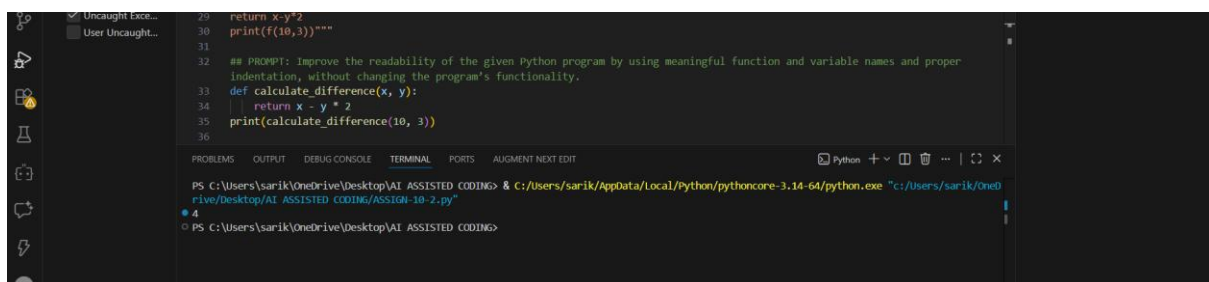
Prompt: Improve the readability of the given Python program by using meaningful function and variable names and proper indentation, without changing the program's functionality.

Corrected Input Code:



```
27 """ TASK-3:
28 def f(x,y):
29     return x-y*2
30     print(f(10,3))"""
31
32 ## PROMPT: Improve the readability of the given Python program by using meaningful function and variable names and proper
33 indentation, without changing the program's functionality.
34 def calculate_difference(x, y):
35     return x - y * 2
36 print(calculate_difference(10, 3))
```

Output:

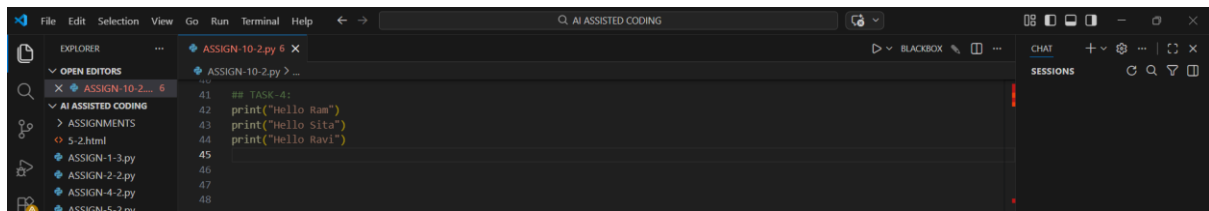


```
29 return x-y*2
30 print(f(10,3))"""
31
32 ## PROMPT: Improve the readability of the given Python program by using meaningful function and variable names and proper
33 indentation, without changing the program's functionality.
34 def calculate_difference(x, y):
35     return x - y * 2
36 print(calculate_difference(10, 3))
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Explanation: The function and variable names were updated to clearly describe their purpose, making the code easier to understand. Proper indentation and clearer expressions were also applied while preserving the original logic.

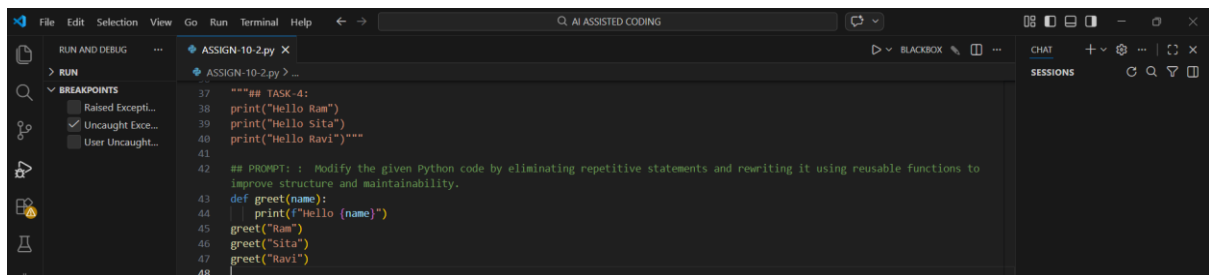
TASK-4: Structural Refactoring

Sample Input Code:

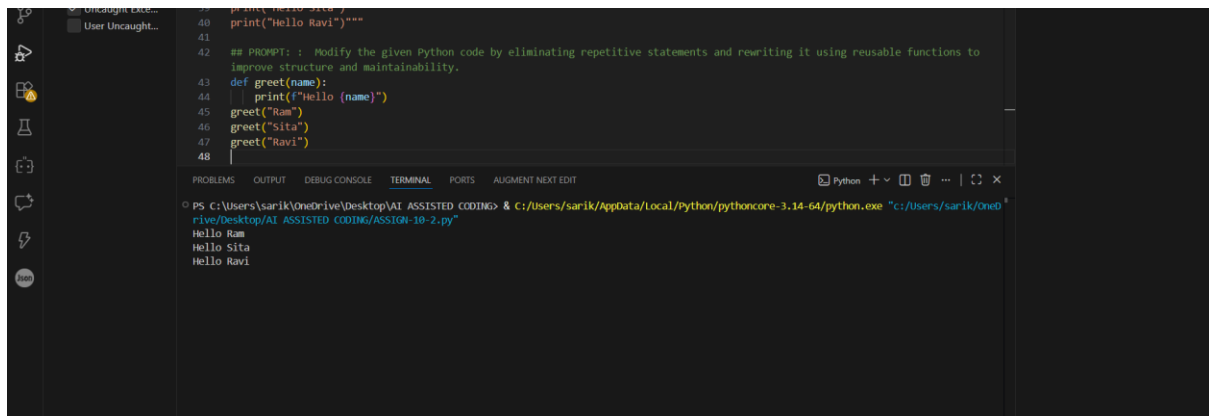


Prompt: Modify the given Python code by eliminating repetitive statements and rewriting it using reusable functions to improve structure and maintainability.

Corrected Input Code:



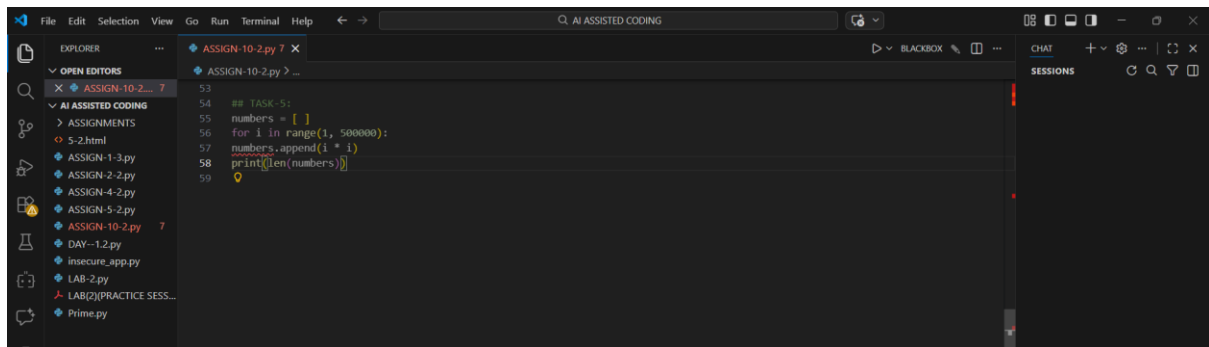
Output:



Explanation: The repetitive print statements were replaced with a reusable function that accepts a name as input. This approach reduces redundancy and makes the code more scalable and easier to modify.

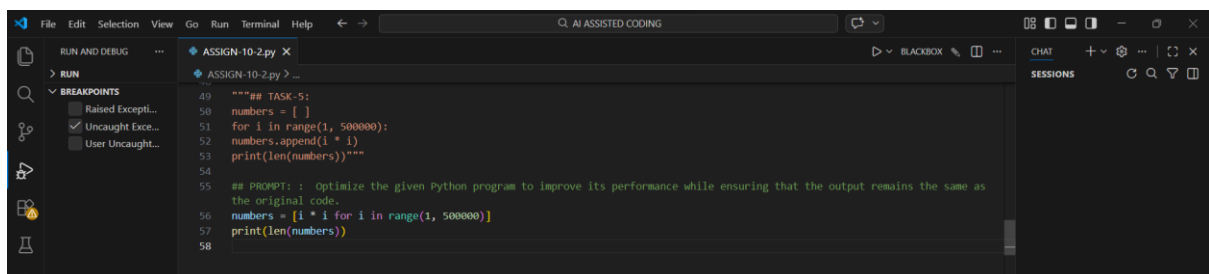
TASK-5: Efficiency Enhancement

Sample Input Code:

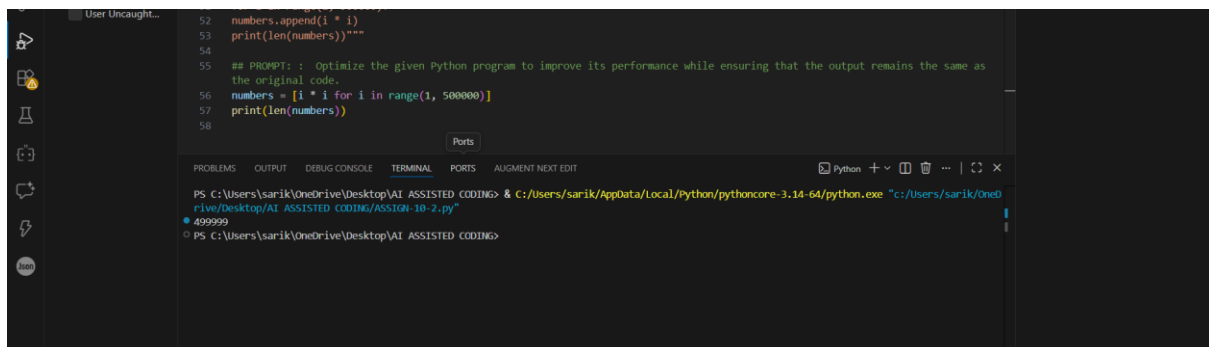


Prompt: Optimize the given Python program to improve its performance while ensuring that the output remains the same as the original code.

Corrected Input Code:



Output:



Explanation: The loop-based implementation was optimized using a list comprehension, which is faster and more concise in Python. This improves performance while producing the same output as the original code.